The Agentic Workflows

How Local Files Are Becoming Departments

Wybe Harms - 2026

The Evolution of Programming

1980s C++

```
#include <iostream>
int main() {
std::cout << "Hi!";
return 0;
}
```

5 lines of ceremony

2010s Python

```
print("Hi!")
```

Just 1 line

2025 AI / Natural Language

"Say hi"

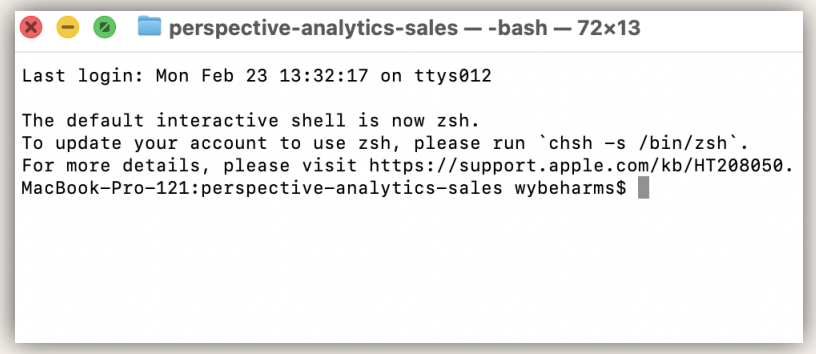
Just speak

Each generation removed a layer of complexity.

Terminal, CLI, GUI

Terminal

Window to type commands



CLI

Typing instructions to terminal

```
$ git push origin main
```

GUI

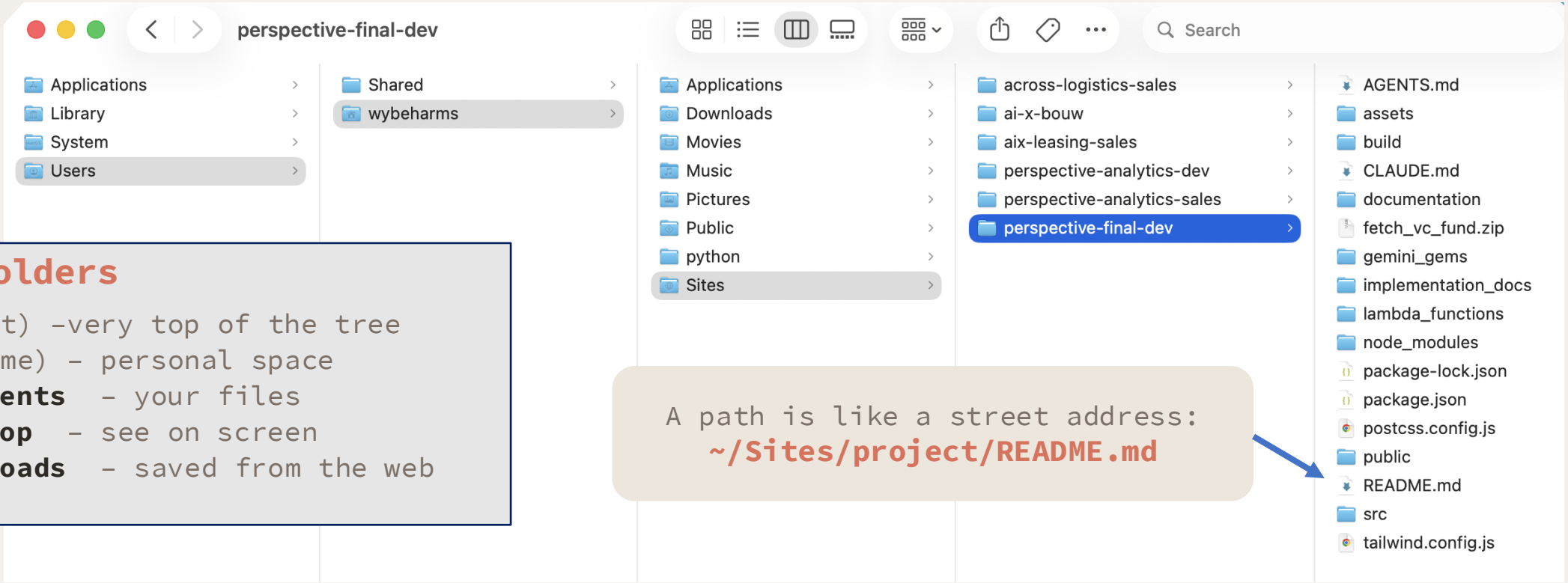
Visual interface

Clicking a button = system running a command for you.

GUIs were built for humans. Agents don't need them.

The File System

A **tree-like structure** of folders and files, starting from a single root directory. Everything on your computer lives somewhere in this tree.



Key Folders

/ (root) -very top of the tree
~/ (home) - personal space
~/Documents - your files
~/Desktop - see on screen
~/Downloads - saved from the web

A path is like a street address:
~/Sites/project/README.md

Agents read, write & navigate the file system using paths

From Chatbot to Agent

2022 — Chatbot

ChatGPT 5.2 Share

example of c++ code

Here's a **simple, clean example of C++ code** to get you started. This is the classic "Hello, World!" program — every beginner should understand this one fully.

C++

```
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
    return 0;
}
```

Text **in**, text **out**

It talks.

2025 — Agent

yes, go ahead

Let me read the full enrich skill first, then make all the changes.

● Read **1** file (ctrl+o to expand)

Now let me read the current state of the files I need to update (some were reverted by the system r

● Read **3** files (ctrl+o to expand)

Good, now I have full context. Let me read the current pipeline.md sections I need to update (check

● Read **1** file (ctrl+o to expand)

Good, I have the full picture. Now let me make all the changes. I'll start with the biggest one — e

● **Update**(.claude/skills/enrich/SKILL.md)
└ Added **13** lines, removed **8** lines

```
3 -description: Enrich prospects with comprehensive research. Batch mode: enriches a staging C
  -le-company mode: research one company and create .md directly.
4 -# allowed-tools: Read, Write, Edit, Glob, Grep, WebSearch, WebFetch, Task, ToolSearch
3 +description: Research and add contacts to the pipeline. Handles all types — cold prospects,
  + Universal entry point for both "add [person]" and "enrich [company]".
4 +# allowed-tools: Read, Write, Edit, Glob, Grep, WebSearch, WebFetch, Task, AskUserQuestion,
```

Instruction **in**, actions **out**

It completes tasks.

Agent: Brain and Hands

Model
(The Brain)

Tools
(The Hands)

File System
(The Body)

*Claude Code is the **"harness"** between model & tool*

Tools — What the Hands Actually Do

The tool is just a programmatic command

File Operations

Read, write, edit, move, rename

```
move file.txt /archive
delete temp.log
```

Shell Commands

Run any terminal command

```
npm install
python script.py
```

Web Search

Search the internet

```
search("OpenAI news")
search("weather NYC")
```

Agent — How It Actually Works

A simple example: *"What files are in my folder?"*

1

Human Asks

"What files are in my folder?"

2

Model Thinks

I need the Bash tool to run “ls”

3

Tool Call

Sends programmatic script to tool

4

Tool Executes

Terminal runs command and returns output

5

Model Responds

"I see two files:

- utils.js”
- main.py

What the model sends →

```
{
  "type": "tool_use",
  "name": "Bash",
  "input": { "command": "ls" }
}
```

← What comes back

```
{
  "type": "tool_result",
  "content": "main.py
utils.js"
}
```

*The brain decides **WHICH** tool to use. The tool executes. The brain interprets the result and communicates with human*

Agent — How It Actually Works

A simple example: *"What files are in my folder?"*

1.

```
MacBook-Pro-121:perspective-analytics-dev wybeharms$ claude

Claude Code v2.1.63
Opus 4.6 · Claude Max
~/Sites/perspective-analytics-dev

› What files are in my folder?

● Bash(ls /Users/wybeharms/Sites/perspective-analytics-dev)
  └ AGENTS.md
    assets
    build
    … +13 lines (ctrl+o to expand)

∴ Thinking…

  The user is asking what files are in their folder. Let me list the files in the cu

  Showing detailed transcript · ctrl+o to toggle · ctrl+e to show all
```

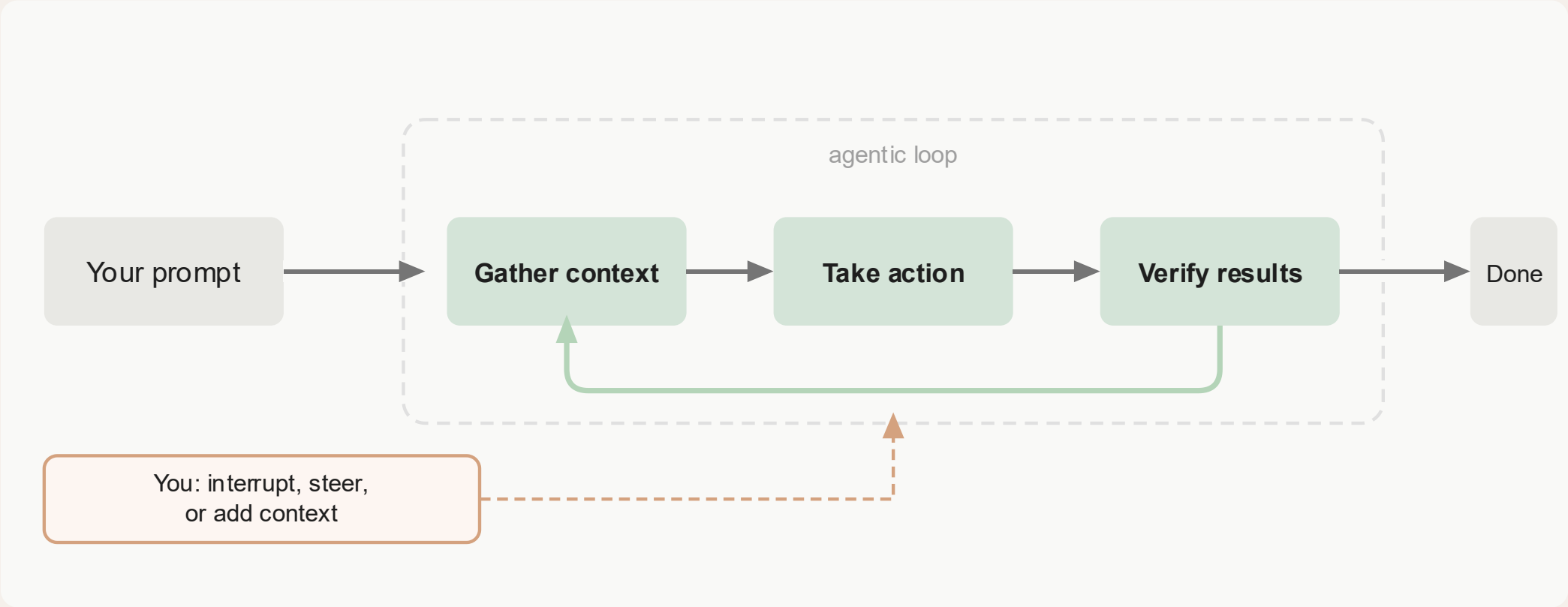
2.

```
Claude Code v2.1.63
Opus 4.6 · Claude Max
~/Sites/perspective-analytics-dev

› What files are in my folder?

● Read 1 file (ctrl+o to expand)

  Here's what's in the project root:
```

Entry	Type
AGENTS.md	File
CLAUDE.md	File
README.md	File
package.json / package-lock.json	NPM config
tailwind.config.js / postcss.config.js	Build config
src/	Frontend source code
public/	Static assets
build/	Production build output
lambda_functions/	AWS Lambda backend code
documentation/	Permanent reference docs
implementation_docs/	Temporary project notes
gemini_gems/	Gemini-related assets
assets/	Project assets
node_modules/	Dependencies

```
  Want me to drill into any specific folder?

✻ Sautéed for 39s

›
```

The Agentic Loop



Repeat — chains dozens of steps per task.

Expanding the Agent's Toolbox

Default Tools

- Shell commands
- Read files
- Write files

MCP

Model Context Protocol

Invented by Anthropic

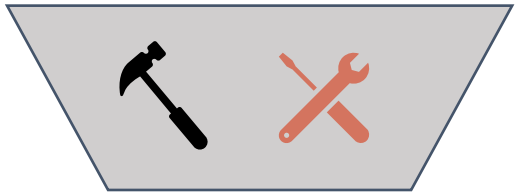
Connects agents to APIs, turning them into tools

Expanded Toolbox

Shell + Read + Write

+ Any API via MCP

Thousands of tools



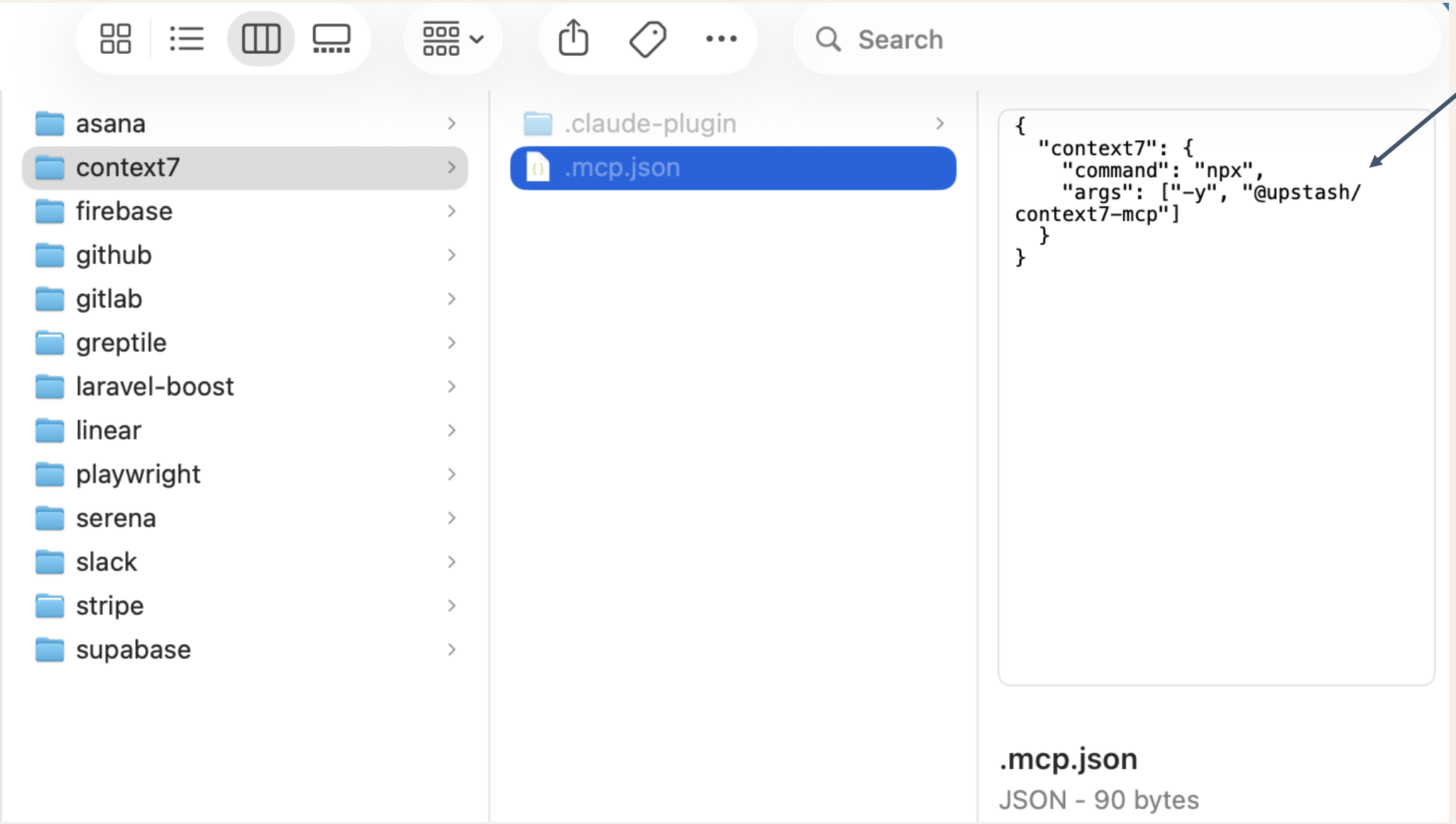
The software world runs on APIs

Stripe API
→ Charge a credit card

Google Maps API
→ Get directions

Slack API
→ Send a message

MCPs are the bridge between agents and APIs. Translate complex API calls into simple instructions the agent can follow



Just a .json file

Popular MCPs – *New MCPs created daily.*

Brave Search
Web search

Firecrawl
Scrape websites

Slack
Read/post messages

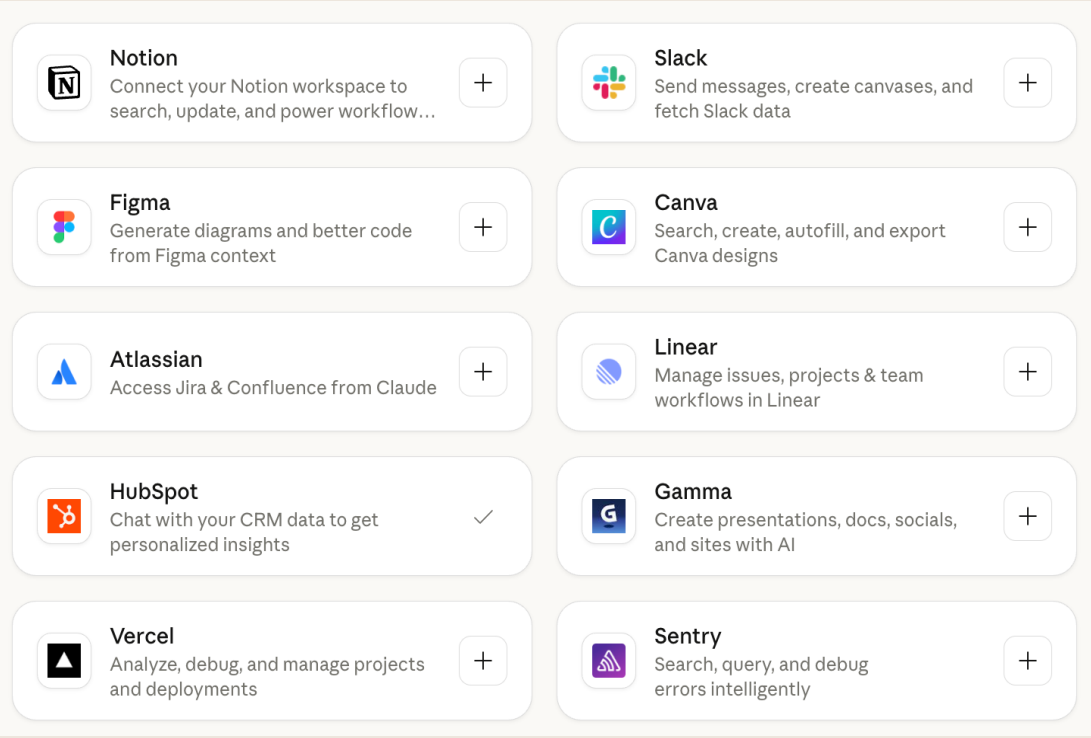
Google Workspace
Docs, Sheets, Gmail

Playwright
Control a browser

Google Maps
Places + directions

Playwright
Control a browser

Google Maps
Places + directions



Recap: Brain and Hands

LLM
(The Brain)

Tools
(The Hands)

File System
(The Body)

Claude Code = “harness”

How do we use these powerful loops to mimic an actual workflow?

SKILL.md

Skills — Guiding the Brain

SKILL.md

.claude > skills > account-research > SKILL.md > # Account Research > ## Output Format

```
---
name: account-research
description: Research a company or person and get actionable sales intel. Works standalone with web search,
supercharged when you connect enrichment tools or your CRM. Trigger with "research [company]", "look up
[person]", "intel on [prospect]", "who is [name] at [company]", or "tell me about [company]".
---

# Account Research

Get a complete picture of any company or person before outreach. This skill always works with web search, and
gets significantly better with enrichment and CRM data.

## How It Works

```
┌──────────────────────────────────────────────────────┐
│                    ACCOUNT RESEARCH                  │
├──────────────────────────────────────────────────────┤
│  ALWAYS (works standalone via web search)            │
│  ✓ Company overview: what they do, size, industry    │
│  ✓ Recent news: funding, leadership changes, announcements │
│  ✓ Hiring signals: open roles, growth indicators     │
│  ✓ Key people: leadership team from LinkedIn         │
│  ✓ Product/service: what they sell, who they serve   │
├──────────────────────────────────────────────────────┤
│  SUPERCHARGED (when you connect your tools)          │
│  + Enrichment: verified emails, phone, tech stack, org chart │
│  + CRM: prior relationship, past opportunities, contacts │
└──────────────────────────────────────────────────────┘
```

Getting Started

Just tell me who to research:

- "Research Stripe"
- "Look up the CTO at Notion"
- "Intel on acme.com"
```

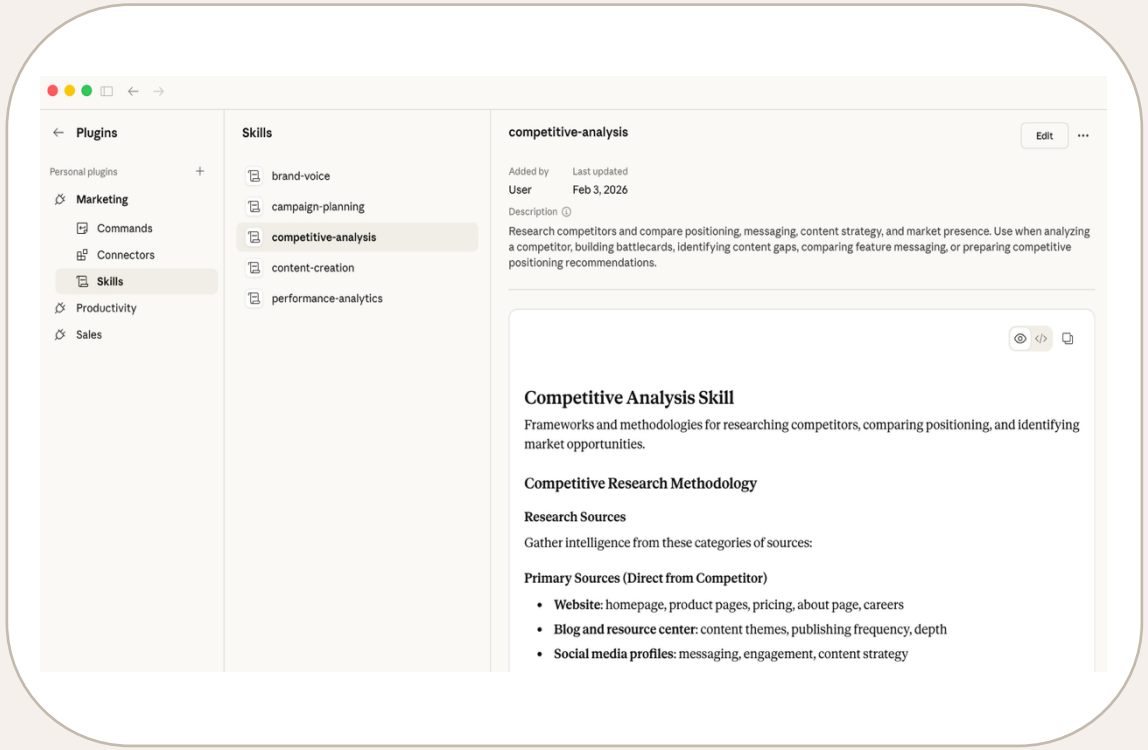
Example skills:

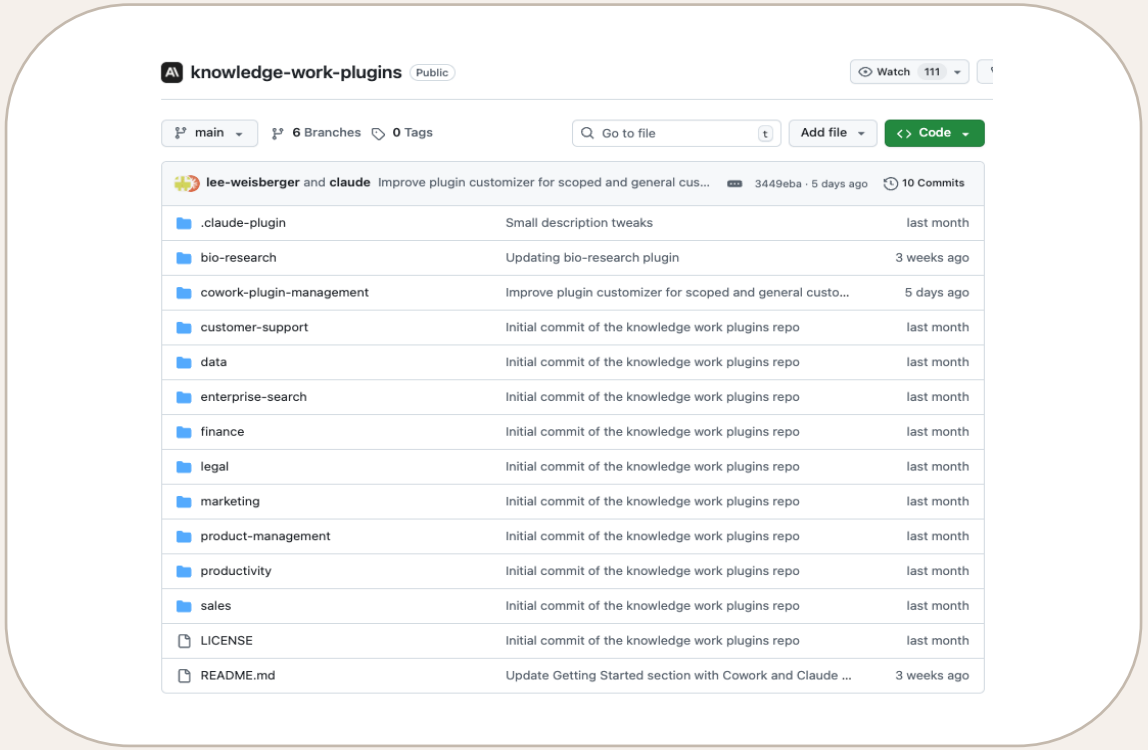
```
/.claude/skills/
  ├── meeting-workflow/SKILL.md   — Prep, process transcript
  ├── manage-pipeline/SKILL.md    — Update CRM, log activities
  ├── crm-check/SKILL.md          — Look up any contact
  ├── source-linkedin/SKILL.md    — Mine LinkedIn for prospect
  ├── create-content/SKILL.md     — Draft blog posts
  ├── weekly-report/SKILL.md      — Generate performance report
```

A skill is a markdown file that teaches the agent how to do something.

Prebuilt Skills

*Anthropic ships **official skills** out of the box*





Skill library in Claude Code

Browse and install skills directly from Cowork

Official skills on GitHub

Open-source, community-driven skill repository

https://github.com/anthropics/skills

Skills People Have Built

SEO Audit

Website optimization

Lead Generation

Find prospects

Video Creation

Programmatic video

Last 30 Days

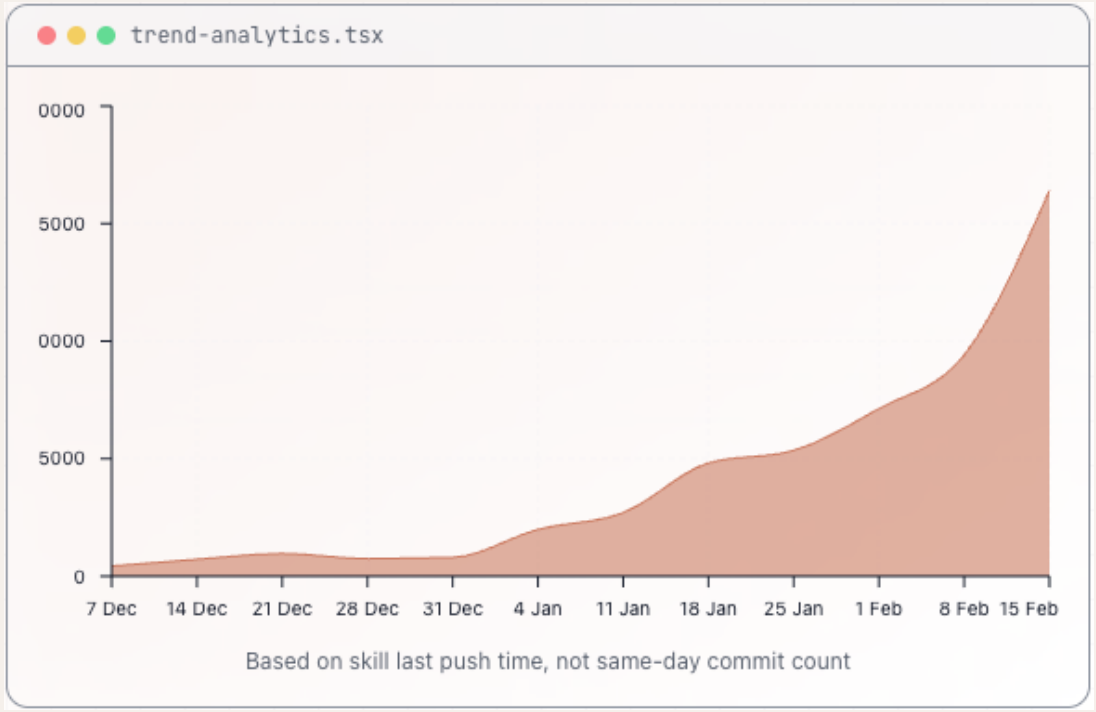
Recent event research

Humanizer

Natural-sounding text

Competitive Analysis

Battlecards



250,000+ community skills.

Example Skills – “account_research”

Research Duke University Endowment Fund

/sales:account-research Research Duke university endowment fund

Updated task list tracking company research progress

Research: Duke University Endowment (DUMAC)

Generated: 2026-03-02 **Sources:** Web Search + Pipeline Check

⚠️ Pipeline Flag

Duke is already in the pipeline as **C0730 — Status: Disqualified**. Prior contact: Mo Bississo (mo.bississo@duke.edu — email invalid). This record was sourced from an FO list and disqualified, but no reason is noted in the CSV. Given the new context below, Wybe may want to reconsider — the ICP fit is stronger than it looks.

Quick Take

DUMAC is a $12.3B endowment manager with ~50 investment professionals processing alternatives across every asset class. They already use Backstop (Clint Coghill's company) and Canoe Intelligence — both of which are direct ICP signals. The credibility angle here is unusually strong. The challenge is size: at this AUM and team depth, they likely have heavy tooling in place. Best entry point is through th ckstop/Clint Coghill connection rather than cold outreach.

Progress 4 of 4

perspective-analytics-sales

Instructions · CLAUDE.md

icp.md

overview.md

Scratchpad

Context

Connectors

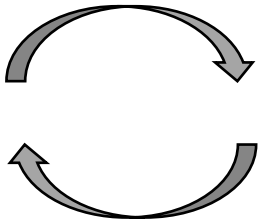
Web search

Skills

account-research

Recap

LLM
(The Brain)



Tools
(The Hands)

Claude Code/Codex/Gemini = “harness”

/Users/wybeharms/Sites/
perspective-analytics

```
perspective-analytics-sales/
├── account-management/
│   ├── 1-cold-prospects/
│   ├── 1-cold-outreach.csv
│   └── ...
├── company-context/
│   ├── overview.md
├── skills/
├── AGENT.md/
└── CLAUDE.md
```

/Users/wybeharms/Sites/
personal_folder

```
personal_folder/
├── wybe/
│   ├── friends/
│   ├── business
│   └── ...
├── hobbies/
│   ├── f1.md
```

/Users/wybeharms/Sites/
example

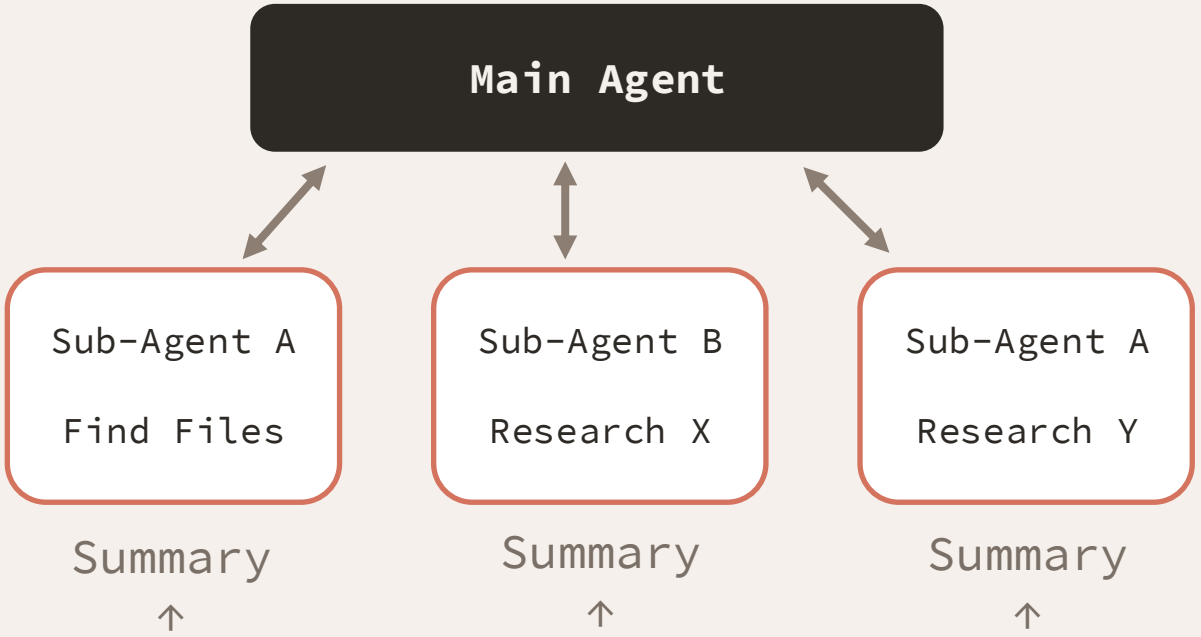
```
example_folder/
├── sales/
│   └── ...
├── finance/
```

Sub-Agents — Delegation

Context window = working memory.

```
/context
Context Usage
claude-opus-4-6 · 144k/200k tokens (72%)

Estimated usage by category
System prompt: 3.5k tokens (1.8%)
System tools: 16.4k tokens (8.2%)
Custom agents: 65 tokens (0.0%)
Memory files: 6.2k tokens (3.1%)
Skills: 1.5k tokens (0.8%)
Messages: 113.9k tokens (57.0%)
Free space: 25k (12.7%)
Autocompact buffer: 33k tokens (16.5%)
```

Orchestrates, stays lightweight



Sub-Agents in Action

1. Let's first fill in the types that are empty. 2. Append research findings. I just want to be very careful that we don't add too much content to the called outreach CSV because I want to limit how many tokens it has. 3. Yes, let's fill in the country if possible. 4. Let's do it step-by-step. Start with three batches. Delegated to the appropriate sub-agents.

Clear.

● Read 1

● Bash(py
imp
└ Erro
Fi

Synt

● Bash(py
imp
└ Tota
P1 (
P2 (
… +5

Good. F

● Bash(py
imp
└ Batc
C0
C0007: PrideCo Capital Management
… +70 lines (ctrl+o to expand)

● 3 Task agents launched (ctrl+o to expand)
├─ Score batch 1 of 25 companies
│ └ Running in the background (shift+↑ to manage)
├─ Score batch 2 of 25 companies
│ └ Running in the background (shift+↑ to manage)
└─ Score batch 3 of 25 companies
└ Running in the background (shift+↑ to manage)

All 3 sub-agents are running in parallel, each researching 25 companies
/tmp/batch2_results.json, and /tmp/batch3_results.json.

This will take a few minutes — each agent needs to do ~25 web searches.

Launching 3 sub-agents in parallel now.

● 3 Task agents launched (ctrl+o to expand)
├─ Score batch 1 of 25 companies
│ └ Running in the background (shift+↑ to manage)
├─ Score batch 2 of 25 companies
│ └ Running in the background (shift+↑ to manage)
└─ Score batch 3 of 25 companies
└ Running in the background (shift+↑ to manage)

All 3 sub-agents are running in parallel, each researching 25 companies via web search. They'll write results to /tmp/batch1_results.json, /tmp/batch2_results.json, and /tmp/batch3_results.json.

This will take a few minutes — each agent needs to do ~25 web searches. I'll check on them when they complete and then merge the results into the CSV.

✻ Crunched for 1m 26s

Claude.md

CLAUDE.md

Boris Cherny @bcherny · Jan 31

3. Invest in your https://t.co/v4FOLUBHz9. After every correction, end with: "Update your https://t.co/v4FOLUBHz9 so you don't make that mistake again." Claude is eerily good at writing rules for itself.

Ruthlessly edit your https://t.co/v4FOLUBHz9 over time. Keep iterating
Show more

```
Memory files · /memory
└ ~/.claude/CLAUDE.md: 76 tokens
└ CLAUDE.md: 4k tokens
```

55 127 3.2K 500K

```
- Use bullet points, abbreviations, and minimal prose.
- At the end of each plan, list unresolved questions to answer, if any.
```

Tips:

- Keep short
  - Added to every chat, so limit tokens
- Every time Claude makes mistake, add correct to Claude.md
- Give context
- Use GitHub saving versions
- Add universal rules

https://x.com/bcherny/status/2017742741636321619

Recap: Everything is a File

```
Soul_spring_project/
  ├── Claude.md        ← The Glue
  ├── Agents.md        ← Agent Defs
  ├── Memory.md        ← Context
  ├── skills/          ← The Brain
  │   ├── account_research.md
  │   └── ...
  ├── mcps/            ← The Hands
  │   └── (tool configs)
  └── agents/          ← Delegation
      └── (sub-agents)
```

Configuration

Claude.md + *Agents.md* + *Memory.md* define identity & context.

Capabilities

skills/ teach how to think.
mcps/ give tools to act.

Scale

agents/ delegate to specialists.
More files = more power.

Just files in a folder — endlessly extensible.

Why Markdown– *Machine Readability*

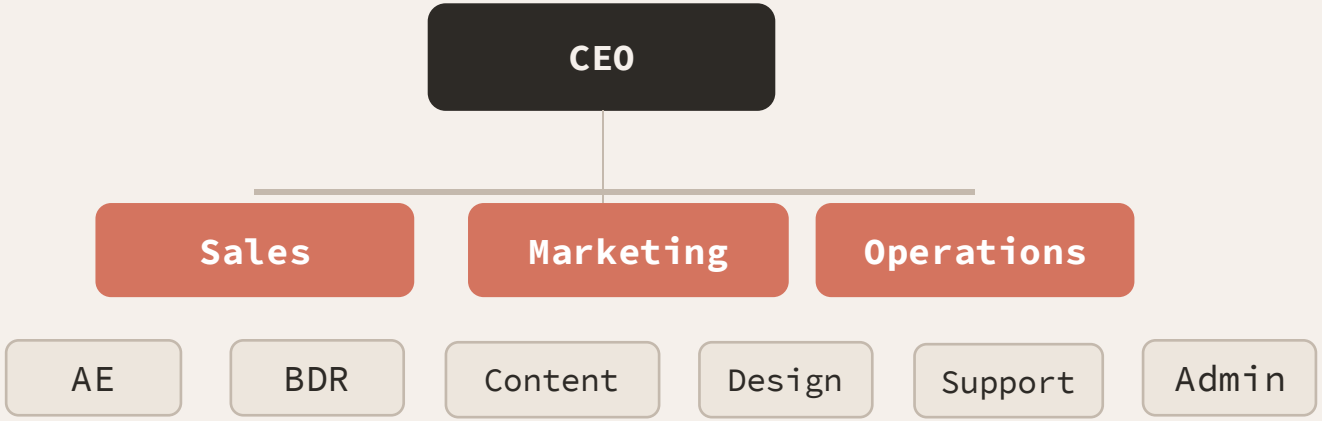
	Markdown (.md)	Word Doc (.docx)	PDF (.pdf)
What it is	Plain text with simple symbols for formatting: # = heading ** = bold	A ZIP file containing complex XML, styles, metadata, and media	A page-rendering format storing exact visual layout and character positions
What a computer sees	Clean, structured text it can read instantly	Nested XML that must be unzipped and parsed	Character coordinates on a canvas — no structure at all
Machine-readable?	**Easy**	**Possible, but messy**	**Very hard**

```
# CLAUDE.md — Perspective Analytics Zero-Person

## Vision

This folder is an entire sales and marketing dep
sales, reporting to the CEO (Wybe). The relation
decisions; Claude takes action across prospectin
meeting support, and executive assistance.

- **Claude handles the heavy lifting.** Sourcing
qualifying leads, drafting outreach, creating co
generating reports, monitoring for pain signals
customer feedback and product insights.
```

```
PK^C^D^T^@^F^@^H^@^@^@!^@éQ^P°<8d>^A^@^@A^E^@^@^S^@^H^
«Ý<8f>ì,_ÿÞi<81>^FIi(ziÒNß÷}fÚ<9d>Þ`©<8b>h^N^^<95>5)ë&
>^@¿îtn¸´&<80>    q(=X¿÷^@¹<98>^U!^Z.éñ<9a><84>ä,º_¿WF
<"<9c><9b>l<8f>.Þ<90>%¤¬Ìqª^\^^]Nøp0ÙKPºl-*^PÕ3<8d>Ó«^
bGÛ<9e>^F!Ð<90>N  ùþÛÆmCÇ<8d>s+Â^BÞ_þ<8d>bÇ¼^U$§ó4^Vï^
êõÐtÄêDZXg÷^GåFÌ kÈæÕ^Fî^?^A^@^@ÿÿ^C^@PK^C^D^T^@^F^@^H
```

```
%PDF-1.4 %□□□□ 1 0 obj << /Type /Catalog /Pa
/PageLayout /OneColumn >> endobj 3 0 obj << /Typ
endobj 9 0 obj << /Producer (EVO HTML to PDF Co
595.00000 842.00000] /Type /Page /Contents [10 0 R
/EvoPdf_eojhdjgfmleokolmpblhodkjpgmihaol 12 0 R
/ProcSet [/PDF /Text /ImageC] /ExtGState << /EvoP
10 0 obj << /Filter /FlateDecode /Length 166 >> strea
□□]O□M[(□□□□□b¸I□^v□□N□`□□T7□□
□□□□y□&Ig□□%1ytJ□\!□□L@e endstream e
x^e□1□□0□□□□□}□□□l□□□CE□□\p
□□1:□`u□□*□o□V□?□□□□□□□□Nz□□
endstream endobj 5 0 obj << /MediaBox [0.00000 0.
```

How Humans Have Always Organized Work



Roles

Who does what

Context

What each person knows

Processes

How tasks get done

Memory

What the org has learned

*Don’t need to **reinvent the wheel***

Human org most proven method for getting things done at scale.

A Folder IS a Department

CRM

Context

Memory

Processes

Roles

```
perspective-analytics-sales/
├── account-management/        ← CRM (data + profiles)
│   ├── 1-cold-prospects/
│   ├── 2-warm-prospects/
│   ├── 1-cold-outreach.csv
│   └── ...
├── company-context/           ← Context (what the business is)
│   ├── overview.md  ├── product.md  ├── icp.md
├── skills/                    ← Standards (how output should look)
├── memory/                    ← Memory (lessons learned)
├── output/                    ← Deliverables (drafts, content)
├── workflows/                 ← Processes (how tasks get done)
├── mcp/                       ← Overview of available tools
├── AGENT.md/                  ← Subagent instructions
└── CLAUDE.md                  ← Operating instructions
```

Growing the Department

Day 1

Files | Shell | Search

Add MCPs, Skills, Agents over time

Month 3

Files | Shell | Search | Maps

Email | Slack | Browser

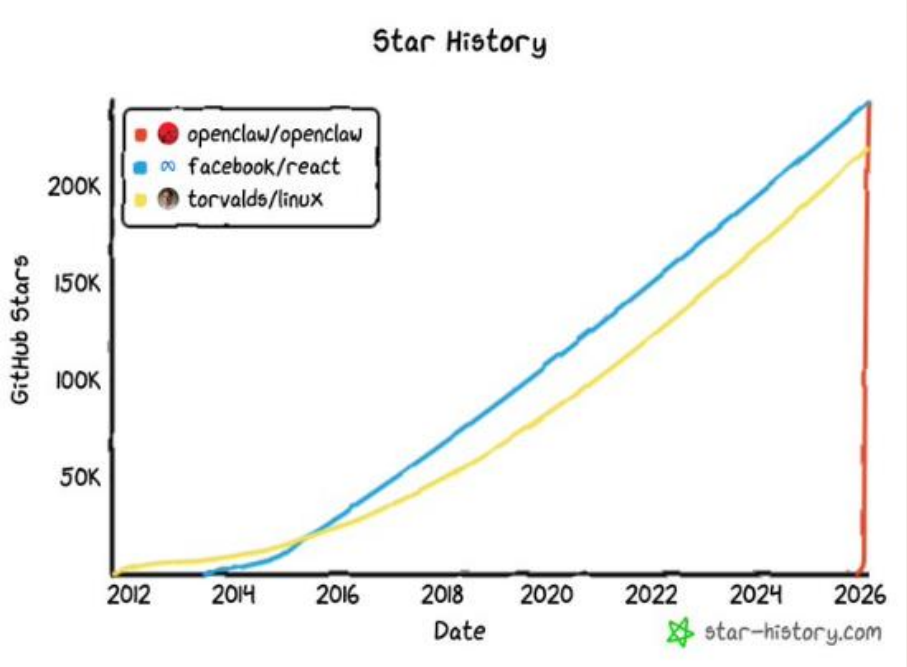
LinkedIn | Intel

Each addition = one more hand. You control what gets added.

Very little moat. Anyone can create markdown files

OpenClaw — A Pre-Built Agent Folder

- 250K+ GitHub stars (popular agent framework)
- Open source, free
- Can run 24/7 (on Mac Mini or VPS)
- Connects to WhatsApp, Telegram, Slack, Discord.




```
~/.openclaw/
    openclaw.json        — Config
    credentials/         — API keys
    sessions/            — Conversation logs
    memory/              — Database + daily
    workspaces/default/
        AGENTS.md        — What to do
        SOUL.md          — How to behave
        USER.md          — User preferences
        MEMORY.md        — Long-term memory
        HEARTBEAT.md     — Proactive checks
        skills/          — Installed skills
```

It's all just markdown files in a folder.

The framework adds structure + channel integrations.

Security — What to Watch For

API Keys

Store in .env files.
Never commit.
Never share.

Prompt Injection

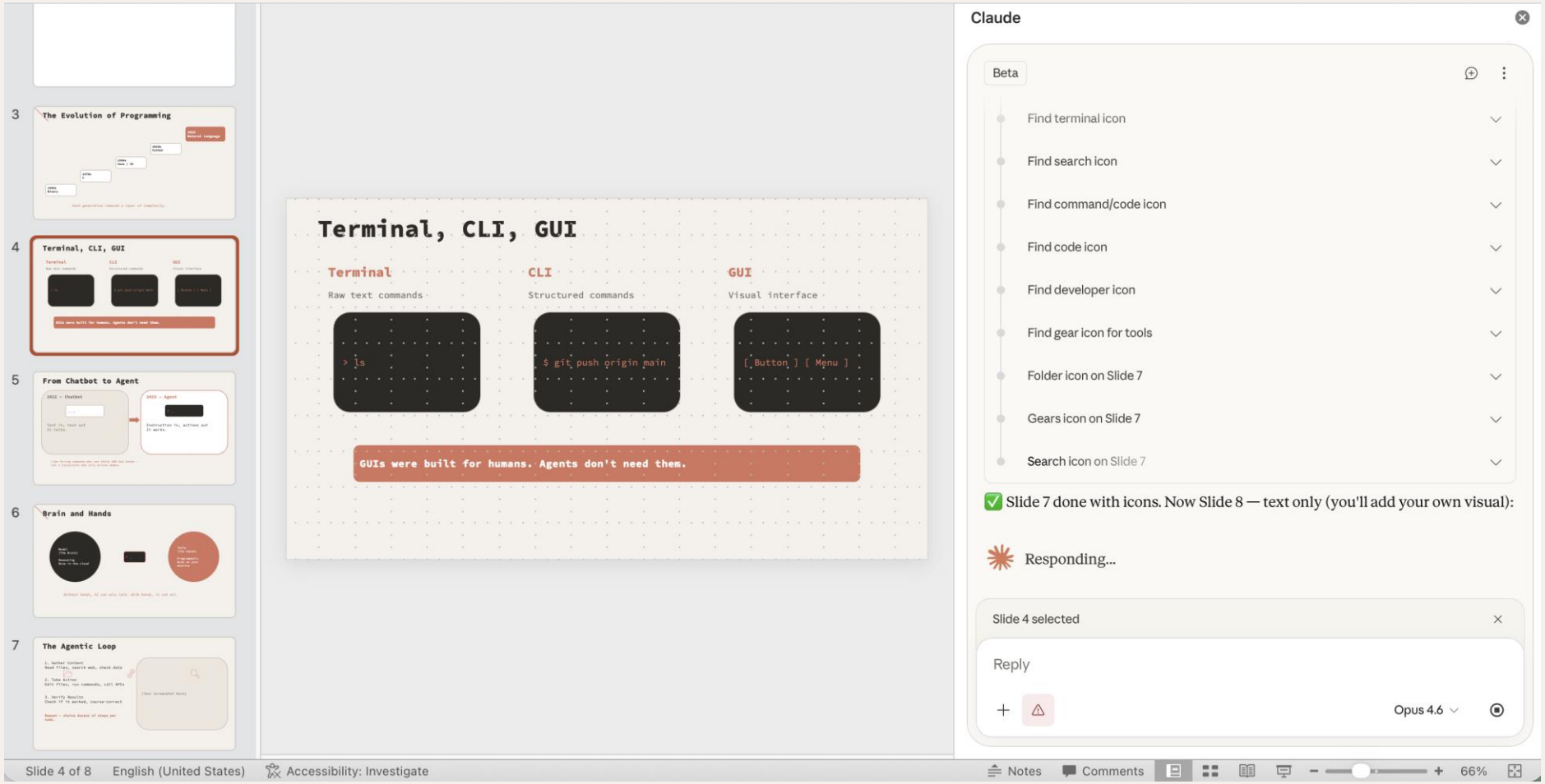
Malicious skills can hide instructions.
Write your own or audit downloads.

Human Approval

Nothing irreversible without human sign-off.

135K+ OpenClaw instances found exposed online. If it runs 24/7, security matters more.

Using Claude Agent to Build PowerPoint



Archive

Why Local Files Beat Cloud

Dimension	Local	Cloud
Context Bloat	Minimal tokens	30K+ per service
Reliability	No network needed	Depends on uptime
Speed	Instant	Round-trip latency
Flexibility	Works with any AI, forever	Proprietary lock-in
Portability	Copy folder → done	Migration pain

Vision: *Local as foundation. Cloud as enhancement.*

The Hidden Cost of Cloud Tools

Local file read

~500 tokens

Cloud API call

~30,000 tokens 60x more!

```
Context Usage Warnings
└ ⚠ Large MCP tools context (~59,601 tokens > 25,000)
  └ MCP servers:
    └ google-workspace: 74 tools (~37,074 tokens)
    └ zerobounce: 15 tools (~7,515 tokens)
    └ firecrawl: 12 tools (~6,012 tokens)
    └ claude_ai_HubSpot: 7 tools (~3,507 tokens)
    └ brave-search: 6 tools (~3,006 tokens)
    └ (2 more servers)
```

Local files = direct relationship between brain and data.
Cloud APIs = layers of noise.

Every MCP adds tokens to the agent's working memory.

More overhead = less accuracy.